

µOS++ Harness / Board Test — Cheat Sheet

- µOS++ Harness / Board Test — Cheat Sheet
 - 1. Layout
 - 2. One-time setup
 - 3. The platform (tests/platforms/<arch>-rpi-zero-2w/)
 - 4. A HARNESS TEST (a suite in tests/sources/)
 - 5. A BOARD TEST (a port test in <port>/test/<board>/)
 - 5b. A HARDWARE board test (no QEMU)
 - 5c. A HARNESS SUITE on a hardware platform
 - 6. Rules you must not break
 - 7. Commands
 - 8. Quick troubleshooting

µOS++ Harness / Board Test — Cheat Sheet

Condensed from WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md. For an architecture repository (like micro-os-plus-iii-aarch32 / -aarch64) that ships its **own** tests under test/<board>/ and is driven by the xPack harness in the SMP kernel's tests/.

1. Layout

Layout (workspace root)

```
~/Work-smp/micro-os-plus-iii/  
├─ micro-os-plus-iii.git/          # SMP kernel (cloned from -smp) ← harness lives here  
│   └─ tests/                    # the xPack harness  
│       └─ package.json          # added by hand (greetings)  
├─ micro-os-plus-iii-devices.git/  
├─ micro-os-plus-iii-<arch>.git/   # the port: test/boards/<board>/, test/<board>/  
├─ micro-os-plus-iii-smp          -> micro-os-plus-iii.git  
└─ micro-os-plus-iii-devices     -> micro-os-plus-iii-devices.git
```

The two symlinks let the port find its kernel/devices siblings.

2. One-time setup

Setup — commands (cwd: ~/Work-smp/micro-os-plus-iii)

```
# cwd: ~/Work-smp/micro-os-plus-iii
# clone
git clone <smp>.git      micro-os-plus-iii.git
git clone <devices>.git  micro-os-plus-iii-devices.git
git clone <arch>.git     micro-os-plus-iii-<arch>.git
ln -s micro-os-plus-iii.git micro-os-plus-iii-smp
ln -s micro-os-plus-iii-devices.git micro-os-plus-iii-devices

# harness
git -C <plain-iii>.git archive HEAD tests | tar -x -C micro-os-plus-iii.git

# C++20
sed -i 's/CMAKE_CXX_STANDARD 23/CMAKE_CXX_STANDARD 20/' micro-os-plus-iii.git/tests/
CMakeLists.txt
```

tests/cmake/tests-main.cmake — pick the port by platform (replaces `add_subdirectory("../")`):

File: [micro-os-plus-iii.git/tests/cmake/tests-main.cmake](#)

```
# micro-os-plus-iii.git/tests/cmake/tests-main.cmake
set (UOS_AARCH32_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch32.git" CACHE PATH "")
set (UOS_AARCH64_DIR "${CMAKE_SOURCE_DIR}/../../micro-os-plus-iii-aarch64.git" CACHE PATH "")
if (PLATFORM_NAME MATCHES "^aarch32")
    add_subdirectory ("${UOS_AARCH32_DIR}" "port-bin")
elseif (PLATFORM_NAME MATCHES "^aarch64")
    add_subdirectory ("${UOS_AARCH64_DIR}" "port-bin")
else ()
    add_subdirectory (".." "top-bin")
endif ()
```

Kernel package.json (the harness greeting reads `../package.json`):

File: [micro-os-plus-iii.git/package.json](#)

```
{ "name": "@micro-os-plus/micro-os-plus-iii-smp", "version": "7.1.0", "license": "MIT" }
```

3. The platform (`tests/platforms/<arch>-rpi-zero-2w/`)

Platform layout

```
cmake/definitions.cmake      # platform macro + toolchain 15.2 guard
cmake/dependencies-folders.cmake # test sources (+ validator xpack)
cmake/platform-library.cmake  # micro-os-plus::platform + ::platform-support
CMakeLists.txt               # executables + add_test
include/cmsis-plus/platform.h # stub (+ OS_USE_SEMIHOSTING_SYSCALLS for AArch32)
src/platform-support.cpp      # hooks + strong main() + exit/syscalls
```

Platform split (see §5) — `platform-library.cmake`:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/platform-library.cmake
# base: flags, linker, port, C library
add_library (platform-<arch>-interface INTERFACE EXCLUDE_FROM_ALL)
target_include_directories (platform-<arch>-interface INTERFACE "include")
target_compile_definitions (platform-<arch>-interface INTERFACE
    "${xpack_platform_compile_definition}" OS_NCPU=${UOS_BOARD_NCPU} OS_USE_SMP_SCHEDULER=1
    TRACE SEMIHOST QEMU_BUILD <arch-macros>)
target_link_options (platform-<arch>-interface INTERFACE
    -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_QEMU}")
target_link_libraries (platform-<arch>-interface INTERFACE micro-os-plus::aarch32) # AArch32
    also: iii-newlib-reent iii-semihosting
add_library (micro-os-plus::platform ALIAS platform-<arch>-interface)

# support: hooks + main, for harness suites only
add_library (platform-<arch>-support-interface INTERFACE EXCLUDE_FROM_ALL)
target_sources (platform-<arch>-support-interface INTERFACE "src/platform-support.cpp")
target_link_libraries (platform-<arch>-support-interface INTERFACE platform-<arch>-interface)
add_library (micro-os-plus::platform-support ALIAS platform-<arch>-support-interface)
```

package.json — add hidden blocks, configs and an action:

File: micro-os-plus-iii.git/tests/package.json

```
// micro-os-plus-iii.git/tests/package.json
"<arch>-actions": { "hidden": true, "actions": {
    "install": [ "xpm install --config {{ configuration.name }}" ] } },
"<arch>-dependencies": { "hidden": true, "devDependencies": {
    "@xpack-dev-tools/<toolchain>": "15.2.1-1.1.1",
    "@xpack-dev-tools/qemu-arm": "9.2.4-1.1" } },

"<arch>-rpi-zero-2w-cmake-gcc-debug": {
    "inherit": [ "<arch>-actions", "cmake-actions", "<arch>-dependencies", "short-win-paths-properties" ],
    "properties": { "buildType": "Debug", "platformName": "<arch>-rpi-zero-2w",
        "toolchainFileName": "<toolchain>.cmake", "shortConfigurationName": "..." } },
"<arch>-rpi-zero-2w-cmake-gcc-release": {
    "inherit": [ "<arch>-rpi-zero-2w-cmake-gcc-debug" ],
    "properties": { "buildType": "MinSizeRel" } },

"test-<arch>-rpi-zero-2w-cmake": [
    "xpm run prepare --config <arch>-rpi-zero-2w-cmake-gcc-debug",
    "xpm run build --config <arch>-rpi-zero-2w-cmake-gcc-debug",
    "xpm run test --config <arch>-rpi-zero-2w-cmake-gcc-debug",
    "xpm run prepare --config <arch>-rpi-zero-2w-cmake-gcc-release",
    "xpm run build --config <arch>-rpi-zero-2w-cmake-gcc-release",
    "xpm run test --config <arch>-rpi-zero-2w-cmake-gcc-release" ]
```

4. A HARNESS TEST (a suite in `tests/sources/`)

A. `tests/sources/<name>/CMakeLists.txt`

File: `micro-os-plus-iii.git/tests/sources/<name>/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/sources/<name>/CMakeLists.txt
cmake_minimum_required (VERSION 3.20)
add_library (test-<name>-interface INTERFACE EXCLUDE_FROM_ALL)
target_include_directories (test-<name>-interface INTERFACE "include")
target_sources (test-<name>-interface INTERFACE src/main.cpp)
add_library (test::<name> ALIAS test-<name>-interface)
```

The test defines `int os_main(int argc, char* argv[])` and returns 0/non-zero.

B. Register the sources — add to the platform `cmake/dependencies-folders.cmake`:

File: `micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake`

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/cmake/dependencies-folders.cmake
"${CMAKE_SOURCE_DIR}/sources/<name>"
```

C. Build + run — add to the platform `CMakeLists.txt`:

File: `micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
if (ENABLE_<NAME>_TEST)
  add_test_executable (<name>-test)
  target_link_libraries (<name>-test PRIVATE
    micro-os-plus::common-options
    test::<name>
    micro-os-plus::iii
    micro-os-plus::platform
    micro-os-plus::platform-support) # harness suites link the SUPPORT
  add_test (NAME "${PLATFORM_NAME}-<name>-test" COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --kernel "${TARGET_FILE:<name>-test}.bin") # AArch32: use the shim instead
  set_tests_properties ("${PLATFORM_NAME}-<name>-test" PROPERTIES TIMEOUT 1200)
endif ()
```

5. A BOARD TEST (a port test in `<port>/test/<board>/`)

The port test ships its **own** strong `main()` + startup hooks, so it links the **base** platform, not the support.

A. The port test (already in the port):

Port test layout

```
<port>/test/<board>/<name>/main.cpp      # defines main, hooks, os_main, RESULT:
<port>/test/<board>/src/test-smp-boot.cpp
<port>/test/<board>/include/              # test-console.hpp, test-smp-boot.hpp
```

B. Build + run — add to the platform CMakeLists.txt:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
set (_port_test_dir "${UOS_AARCH<NN>_DIR}/test/rpi-zero-2w")
add_executable (<name>
    "${_port_test_dir}/<name>/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp")
target_include_directories (<name> PRIVATE "${_port_test_dir}/include")
target_link_libraries (<name> PRIVATE
    micro-os-plus::common-options
    micro-os-plus::iii
    micro-os-plus::platform      # base ONLY – no platform-support
    micro-os-plus::devices)      # only if it touches the SD card
add_test (NAME "${PLATFORM_NAME}-<name>" COMMAND
    qemu-system-aarch64 --machine raspi3b -smp 4 --nographic --serial none
    --semihosting-config enable=on,target=native
    --drive "file=${_disk},if=sd,format=raw"      # only if it touches the SD card
    --kernel "${_shim_img}"                      # AArch32; AArch64 uses the .bin
    --device "loader,file=${TARGET_FILE:<name>}.bin,addr=0x10000")
set_tests_properties ("${PLATFORM_NAME}-<name>" PROPERTIES TIMEOUT 1200)
```

SD-backed tests need a blank image first:

File: micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt

```
# micro-os-plus-iii.git/tests/platforms/aarch32-rpi-zero-2w/CMakeLists.txt
set (_disk "${CMAKE_CURRENT_BINARY_DIR}/<name>.disk.img")
add_custom_command (OUTPUT "${_disk}" COMMAND truncate -s 4G "${_disk}" VERBATIM)
add_custom_target (<name>-disk ALL DEPENDS "${_disk}")
```

Tests that need `micro-os-plus::devices` (the `BOARD_TEST_NEED_DEVICES` list): `sd_test` `smp-mat-sdcard-test` `smp-num-test` `smp-pipeline-test` `usb_test`.

Per-test actions. The VS Code plugin lists xpm **actions**, and the inherited `test` action is one per configuration, so every test shows as `<config>-test`. Give each test a named action in the config (same pattern as the hardware platform, §5b):

```
"actions": {
  "test-mutex-stress": "{{ properties.commandCMakePrepareWithToolchain }}" &&
    {{ properties.commandCMakeBuild }} && cd
    {{ properties.buildFolderRelativePath }} && ctest -V -R mutex-stress",
  "test-smp-pipeline": "{{ properties.commandCMakePrepareWithToolchain }}" &&
    {{ properties.commandCMakeBuild }} && cd
    {{ properties.buildFolderRelativePath }} && ctest -V -R smp-pipeline",
  "test-smp-pro-cons": "{{ properties.commandCMakePrepareWithToolchain }}" &&
    {{ properties.commandCMakeBuild }} && cd
    {{ properties.buildFolderRelativePath }} && ctest -V -R smp-pro-cons"
}
```

aarch32-rpi-zero-2w and aarch64-rpi-zero-2w carry the same three.

5b. A HARDWARE board test (no QEMU)

Some boards have no useful emulator — the Luckfox Lyra is the case: QEMU models none of the RK3506's blocks, so an emulated Lyra would be a generic Cortex-A7 wearing the name. Such a board builds **one image per test** (hwd) and is run on silicon through OpenOCD. In the harness it is a **separate platform** whose CTest entry invokes the port's `hw.sh`.

A. New platform `tests/platforms/<arch>-<board>/:`

- `cmake/definitions.cmake` — select the board **before** the port is added (the port reads `BOARD` to pick its `board.cmake`):

```
set (BOARD "luckfox-lyra" CACHE STRING "" FORCE)
```

- `cmake/platform-library.cmake` — defines the **base** (no `QEMU_BUILD`); a **support** interface is added for the harness suites (§5c):

```
target_compile_definitions (... INTERFACE
  OS_NCPU=${UOS_BOARD_NCPU} OS_USE_SMP_SCHEDULER=1
  TRACE SEMIHOST HW_BUILD __ARM_EABI__ __ARM_ARCH_7A__ _GNU_SOURCE)
target_link_options (... INTERFACE
  -nostartfiles -Wl,--gc-sections "-T${UOS_BOARD_LINKER_HW}")
```

The board tests link the **base only** — they bring their own strong `main()` and hooks. The support (a second strong `main()`, for the harness suites) is a separate interface; see §5c.

- `CMakeLists.txt` — a small helper builds each port test and, opt-in, registers its run. The helper sets what the port's own `test/CMakeLists.txt` would set for an hwd application: output name `<app>-hwd`, runtime dir `<binary>/test`, the port flags, and the base platform only.

```

option (ENABLE_HW_TESTS "Register the hardware tests (needs the board)" OFF)

function (add_hw_test _app)
  cmake_parse_arguments (T "" "SECONDS" "SOURCES;INCLUDES;LIBRARIES" ${ARGN})
  add_executable (${_app} ${T_SOURCES})
  target_include_directories (${_app} PRIVATE ${T_INCLUDES})
  set_target_properties (${_app} PROPERTIES OUTPUT_NAME "${_app}-hwd"
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_BINARY_DIR}/test" SUFFIX "")
  target_link_libraries (${_app} PRIVATE micro-os-plus::iii
    micro-os-plus::platform ${T_LIBRARIES})

  if (ENABLE_HW_TESTS)
    set (_secs "${T_SECONDS}")
    if (_secs STREQUAL "")
      set (_secs 600)          # run budget passed to run-hw.sh
    endif ()
    add_test (NAME "${PLATFORM_NAME}-${_app}"
      COMMAND bash "${_port_hw}" ${_app} ${_secs})
    math (EXPR _timeout "${_secs} + 600")
    set_tests_properties ("${PLATFORM_NAME}-${_app}" PROPERTIES
      ENVIRONMENT "BUILD=${CMAKE_CURRENT_BINARY_DIR}" TIMEOUT ${_timeout})
    endif ()
endfunction ()

# smp_test4: shared test-smp-boot.cpp, no devices.
add_hw_test (smp_test4
  SOURCES "${_port_test_dir}/smp_test4/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp"
  INCLUDES "${_port_test_dir}/include" "${_port_test_dir}/smp_test4")

# smp-mat-sdcard-test: shared test-smp-boot.cpp + devices (SD / FatFs).
add_hw_test (smp-mat-sdcard-test
  SOURCES "${_port_test_dir}/smp-mat-sdcard-test/main.cpp"
    "${_port_test_dir}/src/test-smp-boot.cpp"
  INCLUDES "${_port_test_dir}/include" "${_port_test_dir}/smp-mat-sdcard-test"
  LIBRARIES micro-os-plus::devices-rk3506
  SECONDS 1800)          # the SD solver runs for many minutes

# smp_test5: self-contained (own boot threads) + C++ FatFs + SD host.
add_hw_test (smp_test5
  SOURCES "${_port_test_dir}/smp_test5/main.cpp" ${_fatfs_sources}
  INCLUDES "${_fatfs_dir}" "${_port_test_dir}/smp_test5"
  LIBRARIES micro-os-plus::devices-rk3506
  SECONDS 900)

```

hw.sh runs `run-hw.sh "$BUILD/test" <app>`, so each image must sit in `<binary>/test/` named `<app>-hwd` — hence the output name, runtime dir and cleared suffix. `SECONDS` is the run budget; the CTest `TIMEOUT` is that plus a margin, so CTest never kills a run the runner would still allow.

- `package.json` — one **named action per test**, so the xPack VS Code plugin (which lists actions, not CTest tests) shows each test by name:

```

"actions": {
  "test":
    "cd {{ properties.buildFolderRelativePath }} && ctest -V -LE hw",
  "test-mutex-stress":
    "{{ properties.commandCMakePrepareWithToolchain }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R mutex-stress",
  "test-smp_test4":
    "{{ properties.commandCMakePrepareWithToolchain }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp_test4",
  "test-smp-mat-sdcard-test":
    "{{ properties.commandCMakePrepareWithToolchain }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp-mat-sdcard-test",
  "test-smp_test5":
    "{{ properties.commandCMakePrepareWithToolchain }} &&
    {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }}
    && ctest -V -R smp_test5"
}

```

Every hardware test carries LABELS `hw`, and the generic `test` action runs `ctest -LE hw`: on the Lyra that selects **nothing** (all four tests are hardware tests), so a stray `test` cannot start a board run — use the named `test-*` actions, one per power cycle. The QEMU platforms keep the inherited `test` action unchanged.

Each `test-*` action configures the build tree with `commandCMakePrepareWithToolchain` — the pinned `-D CMAKE_TOOLCHAIN_FILE` — before it builds, so a rebuild picks up the right compiler. A fresh configuration still needs `xpm run install` once.

B. Three hardware quirks. All are read at run time (no rebuild):

- **SWD clock** — `hw.sh` (`UOS_HW_ADAPTER_KHZ`) and `openocd.cfg` (adapter speed), raised from 1000 to 4000 kHz. If you see `Invalid ACK / DEBUG LINK LOST`, drop it back (`UOS_HW_ADAPTER_KHZ=2000`).
- **Console throughput** — the port's `uart.hpp` mirrors the console **line-buffered**: `putc/operator<<` accumulate and emit one `SYS_WRITE0` per line, not one `SYS_WRITEC` per character. A partial line needs `uart::uart1.flush()`.
- **OpenOCD log noise** — `openocd.cfg`'s `examine_secondaries` runs once, silently, wrapped in `log_output /dev/null ... log_output default`, so the `Info : [<target>] hardware has N breakpoints` lines do not shred the console.

C. The strong semihosting paradigm. A hardware run has **no process status**: OpenOCD is a debugger, not a parent. So the verdict is a line the runner greps, and the exit is what *stops* the run. The port's `include/semihosting.hpp` picks the trap by build:

File: micro-os-plus-iii-aarch32.git/include/semihosting.hpp

```

// micro-os-plus-iii-aarch32.git/include/semihosting.hpp
#if defined(QEMU_BUILD)           // emulator: SVC + reason-by-VALUE
#elif defined(SEMIHOST_TRAP_HLT) // Pi under an aarch64 OpenOCD target
#else                             // Lyra: cortex_a -> Angel SVC + reason-by-POINTER
#endif

```

and adds the verdict-and-stop the hardware tests end with:

File: micro-os-plus-iii-aarch32.git/include/semihosting.hpp

```
// micro-os-plus-iii-aarch32.git/include/semihosting.hpp
[[noreturn]] inline void report_result (bool pass) noexcept {
    write_str (pass ? "\nRESULT: PASS\n" : "\nRESULT: FAIL\n"); // the verdict (semihosting)
    pass ? exit_success () : exit_failure ();                    // the stop (SYS_EXIT)
}
```

The strong `_Exit()` (`src/semihosting-exit.cpp`) routes `std::exit()` through the same path. A hardware test ends with `semihosting::report_result (true)` after its checks; every failure path calls `report_result (false)`.

Getting the trap wrong is not a missing console. `hlt #0xF000` is UNDEFINED on ARMv7-A: the word is an Undefined Instruction, so the first character faults and, if the fault handler also prints, the board loops printing fault dumps. The board's `board.cmake` picks HLT vs SVC; the build variant picks value vs pointer.

D. Build and run.

Commands (cwd: `~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests`)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
xpm run install --config aarch32-luckfox-lyra-cmake-gcc-debug
xpm run prepare --config aarch32-luckfox-lyra-cmake-gcc-debug
xpm run build --config aarch32-luckfox-lyra-cmake-gcc-debug

# register the run (needs the board attached; one power cycle per test)
PATH="$PWD/build/aarch32-luckfox-lyra-cmake-gcc-debug/xpacks/.bin:$PATH" \
    cmake -DENABLE_HW_TESTS=ON build/aarch32-luckfox-lyra-cmake-gcc-debug
ctest --test-dir build/aarch32-luckfox-lyra-cmake-gcc-debug -V -R smp_test5

# or by hand
BUILD=$PWD/build/aarch32-luckfox-lyra-cmake-gcc-debug/platform-bin \
    ../../micro-os-plus-iii-aarch32.git/test/boards/luckfox-lyra/hw.sh smp_test5
```

Expected: RESULT: PASS in the OpenOCD log, then `[hw] smp_test5 PASSED`.

One test per power cycle: the secondaries are released once, and no run can un-release them. `run-hw.sh` refuses to run a suite for exactly this reason.

5c. A HARNESS SUITE on a hardware platform

A board test brings its own `main()`; a **harness suite** (from `tests/sources/`) does not. To run one on a hardware platform you add a `platform-support` for the board — a hardware variant of the support the QEMU platforms already have.

A. `src/platform-support.cpp` — startup hooks + a strong `main()`:

File: `micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp`

```
// micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/src/platform-support.cpp
void os_startup_initialize_hardware_early (void) {}
void os_startup_initialize_hardware (void) {
    uart::uart1.init ();
    os_startup_initialize_free_store (__heap_start, __heap_end - __heap_start);
    exception::init ();
    // AFTER the free store: setvbuf() allocates the stdout buffer, and there is
    // no heap before os_startup_initialize_free_store().
    setvbuf (stdout, nullptr, _IOLBF, 0);
}

[[noreturn]] static void harness_main_trampoline (void) {
    // Opens the semihosting fds (initialise_monitor_handles) so the C library's
    // _write finds fd 1; without it every printf in the suite is discarded.
    int argc = 0;
    char** argv = nullptr;
    os_startup_initialize_args (&argc, &argv);

    int code = os_main (argc, argv);           // no command line -> argc <= 1
    uart::uart1 << (code == 0 ? "\nRESULT: PASS\n" : "\nRESULT: FAIL\n");
    uart::uart1.flush ();                     // the verdict the runner greps
    std::exit (code);                         // -> strong _Exit -> SYS_EXIT
}
```

The suites print through the C library (`printf`). Its `_write` needs fd 1 in the semihosting table, which `os_startup_initialize_args()` fills (via `initialise_monitor_handles()`); without that call every `printf` is dropped with `EBADF`, while the kernel's own unbuffered trace (the `./!` tick markers) still appears.

B. `cmake/platform-library.cmake` — a support interface that adds the hooks and the kernel's C library (the suites use `printf`, `gettimeofday`):

File: `micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake`

```
# micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/cmake/platform-library.cmake
add_library (platform-<board>-support-interface INTERFACE EXCLUDE_FROM_ALL)
target_sources (platform-<board>-support-interface INTERFACE "src/platform-support.cpp")
target_link_libraries (platform-<board>-support-interface INTERFACE
    platform-<board>-interface
    micro-os-plus::iii-newlib-reent micro-os-plus::iii-semihosting)
add_library (micro-os-plus::platform-support ALIAS platform-<board>-support-interface)
```

`platform.h` must define `OS_USE_SEMIHOSTING_SYSCALLS` (plus `__disable_irq`), and the base needs `_GNU_SOURCE` (the syscall layer uses `S_IREAD`).

C. The suite target — link the base + **support** + the suite + common-options:

File: `micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/CMakeLists.txt`

```
# micro-os-plus-iii.git/tests/platforms/aarch32-luckfox-lyra/CMakeLists.txt
add_executable (mutex-stress-test)
target_link_libraries (mutex-stress-test PRIVATE
    micro-os-plus::common-options test::mutex-stress
    micro-os-plus::iii micro-os-plus::platform micro-os-plus::platform-support)
set_target_properties (mutex-stress-test PROPERTIES
    OUTPUT_NAME "mutex-stress-test-hwd"
    RUNTIME_OUTPUT_DIRECTORY "${CMAKE_CURRENT_BINARY_DIR}/test" SUFFIX "")
```

Add `sources/mutex-stress` to `dependencies-folders.cmake`, register the run like any board test, and add a `test-mutex-stress` action.

The suite source is unchanged: it returns a code from `os_main()`, and the **support trampoline** turns that into the `RESULT:` line — so the same source runs under QEMU (exit code) and on the board (log line).

6. Rules you must not break

Rule	Why
One <code>micro-os-plus::iii</code> — add only the port; never the kernel too	duplicate alias fails
Platform split: base = flags/port/linker; support = hooks/main + kernel C-lib	port tests bring their own <code>main</code> ; two strong mains collide
Harness suite links base + support ; board test links base only	as above
QEMU_BUILD on the platform	AArch32 port selects SVC + reason-by-value; else QEMU exits 1
AArch32 links the kernel's <code>iii-newlib-reent</code> + <code>iii-semihosting</code> ; AArch64 does not (port provides its own HLT)	the kernel groups are AArch32-only
OS_USE_SEMIHOSTING_SYSCALLS in <code>platform.h</code> (AArch32)	else <code>c-syscalls-semihosting.cpp</code> compiles to nothing
C++20	harness/port standard
Unique test names + project name per platform (<code>\${PLATFORM_NAME}-...</code>)	runners cannot tell aarch32 from aarch64 otherwise
install → prepare → build → test	<code>build/<config>/</code> holds the pinned toolchain
arm-none-eabi-gcc 15.2 guard	the system 16.2 must not be picked
Hardware board: no <code>QEMU_BUILD</code> , define <code>HW_BUILD</code> , run via <code>hw.sh</code> ; the verdict is the <code>RESULT:</code> line, not an exit code	OpenOCD has no process status
Harness suite on hardware: needs a board <code>platform-support</code> (+ <code>iii-newlib-reent/iii-semihosting</code>); board tests link the base only	the suite has no <code>main()</code> /hooks

7. Commands

Commands (cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests)

```
# cwd: ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests
cd ~/Work-smp/micro-os-plus-iii/micro-os-plus-iii.git/tests

# first time
npm install
xpm install
xpm run install --config <arch>-rpi-zero-2w-cmake-gcc-debug

# lifecycle
xpm run prepare --config <arch>-rpi-zero-2w-cmake-gcc-debug
xpm run build --config <arch>-rpi-zero-2w-cmake-gcc-debug
xpm run test --config <arch>-rpi-zero-2w-cmake-gcc-debug

# debug + release
xpm run test-<arch>-rpi-zero-2w-cmake

# one test only (xpm's PATH provides qemu-system-aarch64)
B=build/<arch>-rpi-zero-2w-cmake-gcc-debug
PATH="$PWD/$B/xpacks/.bin:$PWD/xpacks/.bin:$PATH" \
  ctest --test-dir "$B" -R <name> -V

# list registered tests
PATH="$PWD/$B/xpacks/.bin:$PATH" ctest --test-dir "$B" -N
```

Expected:

Output

```
1/3 Test #1: aarch32-rpi-zero-2w-mutex-stress-test ... Passed
2/3 Test #2: aarch32-rpi-zero-2w-smp-pipeline-test ... Passed
3/3 Test #3: aarch32-rpi-zero-2w-smp-pro-cons-test ... Passed
100% tests passed, 0 tests failed out of 3
```

8. Quick troubleshooting

Symptom	Fix
alias <code>micro-os-plus::iii</code> already exists	add only the port
undefined reference to <code>os_startup_initialize_hardware*</code>	harness suite must link <code>platform-support</code>
multiple definition of <code>_write/...</code> (AArch64)	the port already defines them; don't redefine
undefined reference to <code>_write/_gettimeofday</code>	AArch32: link <code>iii-newlib-reent</code> + <code>iii-semihosting</code>

Symptom	Fix
test hangs after <code>scheduler::start()</code>	strong <code>main()</code> must set the interrupts stack
<code>***Failed, QEMU exit 1</code>	AArch32: use <code>QEMU_BUILD (SVC + value)</code>
toolchain guard fires (16.2)	<code>rm -rf build/<config></code> then <code>install</code> → <code>prepare</code> → <code>build</code>
<code>test_smpl</code> -era path errors	apps live in <code><port>/test/<board>/</code> , runners in <code>test_smpl/</code>
hardware run faults / no output	wrong trap: HLT for the Pi, SVC for the Lyra; never <code>hlt</code> on ARMv7-A
hardware test never ends (TIMEOUT)	the test must call <code>semihosting::report_result()</code> ; an infinite demo has no verdict
hardware test killed at the budget	raise its <code>SECONDS</code> in <code>add_hw_test</code> (e.g. <code>smp-mat-sdcard-test = 1800</code>)
<code>S_IREAD</code> was not declared building <code>c-syscalls-semihosting.cpp</code>	the base needs <code>_GNU_SOURCE</code>

Companion to *WORK-SMP-AARCH32-AARCH64-HARNESS-GUIDE.md* (full detail) and *TESTS-XPACK-SYSTEM.md* (upstream harness).